

Python Programming

Session 4: (Object-Oriented Programming (OOP))

Agenda:

1	Introduction to OOP Concepts
2	Defining Classes and Creating Objects
3	Practical Examples

Introduction to OOP Concepts

Introduction to OOP Concepts

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects" that can contain data and methods to manipulate that data. This session covers the fundamental concepts of OOP, defining classes and creating objects, followed by practical examples.



Introduction to OOP Concepts

Key OOP Concepts

- **Classes:** A class is a blueprint for creating objects. It defines the attributes (data) and methods (functions) that the objects created from the class will have.
- **Objects:** An object is an instance of a class. It represents a specific example of the class with actual data and behavior.
- **Attributes:** Attributes are variables that belong to a class. They define the properties or state of an object.
- **Methods:** Methods are functions defined within a class that describe the behaviors or actions of the objects created from the class.

Defining Classes and Creating Objects

Defining Classes and Creating Objects

Basic Class and Object Creation

```
class Person:
    # Constructor method to initialize attributes
    def __init__(self, name, age):
        self.name = name # Attribute 'name'
        self.age = age   # Attribute 'age'

    # Method to display the person's details
    def display_info(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Creating an object of the Person class
person1 = Person("Alice", 30)

# Accessing object attributes and methods
print(person1.name) # Output: Alice
print(person1.age)  # Output: 30
person1.display_info() # Output: Name: Alice, Age: 30
```

```
Alice
30
Name: Alice, Age: 30
```

- **Class Definition:** class Person: defines a class named Person.
- **Constructor Method:** __init__(self, name, age) initializes the object's attributes (name and age) when a new Person object is created.
- **Method:** display_info() is a method that prints the object's name and age.
- **Object Creation:** person1 = Person("Alice", 30) creates a new Person object named person1 with the name "Alice" and age 30.

Defining Classes and Creating Objects

Using Multiple Objects

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says woof!")

# Creating multiple Dog objects
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Lucy", "Labrador")

# Calling methods on different objects
dog1.bark() # Output: Buddy says woof!
dog2.bark() # Output: Lucy says woof!
```

```
Buddy says woof!
Lucy says woof!
```

- **Class Definition:** class Dog: defines a class named Dog.
- **Attributes:** name and breed are initialized in the constructor.
- **Method:** bark() is a method that prints a message using the object's name.
- **Multiple Objects:** dog1 and dog2 are different instances of the Dog class, each with its own name and breed.

Defining Classes and Creating Objects

Adding Behavior with Methods

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def start(self):
        print(f"The {self.year} {self.make} {self.model} is starting.")

    def stop(self):
        print(f"The {self.year} {self.make} {self.model} is stopping.")

# Creating an object of the Car class
my_car = Car("Toyota", "Camry", 2020)

# Calling methods on the object
my_car.start() # Output: The 2020 Toyota Camry is starting.
my_car.stop() # Output: The 2020 Toyota Camry is stopping.
```

```
The 2020 Toyota Camry is starting.
The 2020 Toyota Camry is stopping.
```

- **Class Definition:** class Car: defines a class named Car.
- **Attributes:** make, model, and year are initialized in the constructor.
- **Methods:** start() and stop() provide behavior for starting and stopping the car.
- **Object Creation:** my_car = Car("Toyota", "Camry", 2020) creates a Car object with specific details.

Defining Classes and Creating Objects

Updating Object Attributes

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def update_grade(self, new_grade):
        self.grade = new_grade

    def display_info(self):
        print(f"Student: {self.name}, Grade: {self.grade}")

# Creating a Student object
student1 = Student("John", "B")
student1.display_info() # Output: Student: John, Grade: B

# Updating the student's grade
student1.update_grade("A")
student1.display_info() # Output: Student: John, Grade: A
```

Student: John, Grade: B
Student: John, Grade: A

- **Method to Update Attributes:** `update_grade(self, new_grade)` changes the object's grade attribute.
- **Object Creation:** `student1 = Student("John", "B")` creates a Student object.
- **Updating Attributes:** The `update_grade()` method is called to change the student's grade.

Defining Classes and Creating Objects

Using a Method to Return Information

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def get_description(self):
        return f'{self.title}' by {self.author}"

# Creating a Book object
book1 = Book("1984", "George Orwell")

# Using a method to get information about the book
description = book1.get_description()
print(description) # Output: '1984' by George Orwell
```

→ '1984' by George Orwell

- **Method Returning Information:** `get_description(self)` returns a formatted string containing the book's title and author.
- **Object Creation:** `book1 = Book("1984", "George Orwell")` creates a Book object.
- **Calling Method:** The `get_description()` method is called to get a string describing the book.

Summary

- **Classes and Objects:** Classes define the blueprint for objects, and objects are instances of classes.
- **Attributes and Methods:** Attributes store the state of an object, and methods define the behavior of an object.
- **Creating Objects:** Objects are created using the class name followed by parentheses, which call the class constructor.
- **Accessing Methods and Attributes:** Use dot notation (object.method() or object.attribute) to call methods and access or modify attributes.

Practical Examples

Managing a Simple Inventory System

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

    def restock(self, amount):
        self.quantity += amount
        print(f"Restocked {self.name}. New quantity: {self.quantity}")

    def sell(self, amount):
        if amount <= self.quantity:
            self.quantity -= amount
            print(f"Sold {amount} of {self.name}. Remaining quantity: {self.quantity}")
        else:
            print("Not enough stock to sell.")

# Example usage
product1 = Product("Laptop", 1200, 10)
product1.restock(5)    # Output: Restocked Laptop. New quantity: 15
product1.sell(3)       # Output: Sold 3 of Laptop. Remaining quantity: 12
product1.sell(20)      # Output: Not enough stock to sell.
```

Restocked Laptop. New quantity: 15
Sold 3 of Laptop. Remaining quantity: 12
Not enough stock to sell.

- The Product class models a product in an inventory system, with attributes for name, price, and quantity. It includes methods to restock and sell the product, which update the quantity accordingly.

Simple Calculator Class

```
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b != 0:
            return a / b
        else:
            return "Error: Division by zero."

# Example usage
calc = Calculator()
print(calc.add(10, 5))      # Output: 15
print(calc.subtract(10, 5)) # Output: 5
print(calc.multiply(10, 5)) # Output: 50
print(calc.divide(10, 2))   # Output: 5.0
print(calc.divide(10, 0))   # Output: Error: Division by zero.
```

```
15
5
50
5.0
Error: Division by zero.
```

- The Calculator class defines methods for basic arithmetic operations: add, subtract, multiply, and divide. It checks for division by zero to prevent errors.

Creating a Student Management System

```
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.courses = []

    def enroll(self, course):
        self.courses.append(course)
        print(f"{self.name} has enrolled in {course}")

    def display_info(self):
        print(f"Student: {self.name}, ID: {self.student_id}, Courses: {' '.join(self.courses)}")

# Example usage
student1 = Student("John Doe", "S12345")
student1.enroll("Math")
student1.enroll("Science")
student1.display_info() # Output: Student: John Doe, ID: S12345, Courses: Math, Science
```

```
John Doe has enrolled in Math
John Doe has enrolled in Science
Student: John Doe, ID: S12345, Courses: Math, Science
```

- The Student class allows creating student objects with a name and ID. The enroll method adds courses to the student's list, and display_info shows the student's details.

Modeling a Bank Account

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount}. Remaining balance: {self.balance}")
        else:
            print("Insufficient funds.")

# Example usage
account = BankAccount("Alice")
account.deposit(1000) # Output: Deposited 1000. New balance: 1000
account.withdraw(500) # Output: Withdrew 500. Remaining balance: 500
account.withdraw(1000) # Output: Insufficient funds.
```

```
Deposited 1000. New balance: 1000
Withdrew 500. Remaining balance: 500
Insufficient funds.
```

- The BankAccount class simulates a bank account with methods to deposit and withdraw money, while ensuring that withdrawals do not exceed the account balance.

Modeling a Simple Library System

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def display(self):
        print(f"Title: {self.title}, Author: {self.author}")

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print(f"Added book: {book.title}")

    def list_books(self):
        print("Library books:")
        for book in self.books:
            book.display()

# Example usage
book1 = Book("1984", "George Orwell")
book2 = Book("To Kill a Mockingbird", "Harper Lee")
library = Library()
library.add_book(book1) # Output: Added book: 1984
library.add_book(book2) # Output: Added book: To Kill a Mockingbird
library.list_books()
```

```
Added book: 1984
Added book: To Kill a Mockingbird
Library books:
Title: 1984, Author: George Orwell
Title: To Kill a Mockingbird, Author: Harper Lee
```

- The Book class represents a book with a title and author. The Library class manages a collection of books, allowing books to be added and listed.

Simple Voting System

```
class Candidate:
    def __init__(self, name):
        self.name = name
        self.votes = 0

    def add_vote(self):
        self.votes += 1

    def display(self):
        print(f"Candidate: {self.name}, Votes: {self.votes}")

# Example usage
candidate1 = Candidate("Alice")
candidate2 = Candidate("Bob")

# Voting
candidate1.add_vote()
candidate1.add_vote()
candidate2.add_vote()

# Display results
candidate1.display() # Output: Candidate: Alice, Votes: 2
candidate2.display() # Output: Candidate: Bob, Votes: 1
```

```
Candidate: Alice, Votes: 2
Candidate: Bob, Votes: 1
```

- The Candidate class models a candidate in a voting system, tracking the number of votes received. The add_vote method increments the vote count, and the display method shows the candidate's details.

Simple Temperature Converter

```
class TemperatureConverter:
    def __init__(self, celsius):
        self.celsius = celsius

    def to_fahrenheit(self):
        return (self.celsius * 9/5) + 32

    def to_kelvin(self):
        return self.celsius + 273.15

# Example usage
temp = TemperatureConverter(25)
print("Fahrenheit:", temp.to_fahrenheit()) # Output: Fahrenheit: 77.0
print("Kelvin:", temp.to_kelvin())         # Output: Kelvin: 298.15
```

```
Fahrenheit: 77.0
Kelvin: 298.15
```

- The TemperatureConverter class converts a temperature in Celsius to Fahrenheit and Kelvin using methods to_fahrenheit and to_kelvin.

Modeling a Car Rental System

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.is_rented = False

    def rent(self):
        if not self.is_rented:
            self.is_rented = True
            print(f"{self.make} {self.model} rented.")
        else:
            print(f"{self.make} {self.model} is already rented.")

    def return_car(self):
        if self.is_rented:
            self.is_rented = False
            print(f"{self.make} {self.model} returned.")
        else:
            print(f"{self.make} {self.model} was not rented.")

# Example usage
car1 = Car("Toyota", "Corolla", 2020)
car1.rent()      # Output: Toyota Corolla rented.
car1.rent()      # Output: Toyota Corolla is already rented.
car1.return_car() # Output: Toyota Corolla returned.
car1.return_car() # Output: Toyota Corolla was not rented.
```

```
Toyota Corolla rented.
Toyota Corolla is already rented.
Toyota Corolla returned.
Toyota Corolla was not rented.
```

- The Car class models a car in a rental system, with methods to rent and return the car, managing its availability state.